第一阶段：DeepAgents 框架定位与核心价值

学习目标：理解 DeepAgents 是什么、解决什么问题、以及适用场景

目 录目 录

一、什么是 DeepAgents？

二、解决的核心问题

三、应用场景

四、本章小结

一、什么是 DeepAgents？

1.1 定义

DeepAgents（代码库名为 deepagents）是一个基于 LangChain 和 LangGraph 构建的企业级高级智能体框架。

定位：高阶的 Agent Harness（智能体装备/套件）

构建基础：

- 底层运行时：LangGraph
- 工具/模型层：LangChain

1.2 核心理念

如果说 LangChain 提供了积木，LangGraph 提供了地基，那么 DeepAgents 就是一套成品级的框架。

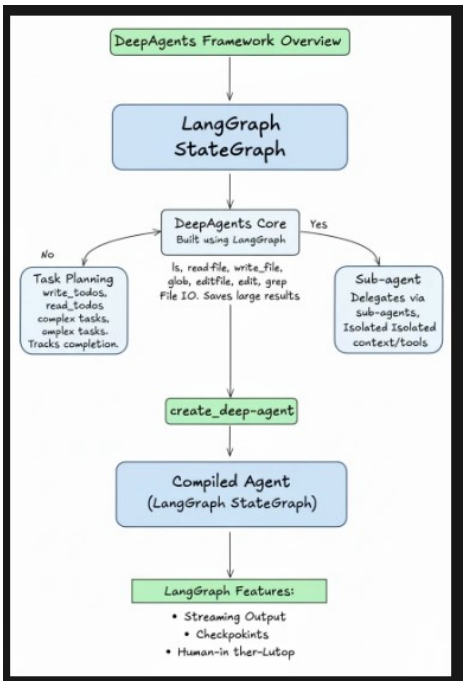
它预设了最佳实践（规划、文件系统、子智能体），让你能快速构建类似 OpenAI Deep Research 或 Claude Code 的应用。

1.3 核心目标

简化长运行自主智能体（Long-running Autonomous Agents）的开发过程，通过内置的最佳实践和中间件，解决复杂任务中的：

- 规划（Planning）
- 记忆（Memory）
- 工具使用（Tool Use）
- 环境交互（Environment Interaction）

1.4 框架架构图



1.5 参考资源

资源	地址
GitHub	https://github.com/langchain-ai/deepagents
论文	https://arxiv.org/pdf/2510.21618

二、解决的核心问题

2.1 传统 Agent 的困境

传统的 Agent 开发运行一个简单的循环：

思考 → 调用工具 → 观察 → 重复

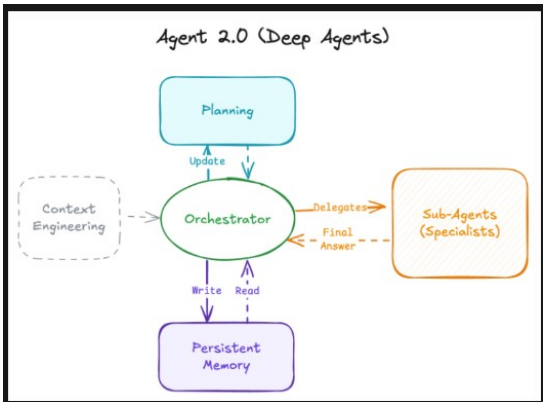
这种模式在处理多小时或多天的任务时，容易遇到以下"浅层陷阱"（Shallow Agent Problem）：

2.2 五大核心问题

序号	问题	描述
1	规划能力缺失	原生 Agent 倾向于"走一步看一步"，缺乏全局

		视角的任务拆解，容易在多步任务中迷失方向
2	遗忘与混乱	在执行超过 10-20 步的长任务时，由于 Context Window（上下文窗口）限制，传统 Agent 容易忘记初始目标或陷入死循环
3	环境交互困难	文件系统操作、代码执行环境（沙箱）的配置和安全管理复杂
4	上下文污染	所有工具返回结果都堆积在一个对话历史中，导致噪声过大
5	协作编排复杂	多智能体（Multi-Agent）之间的任务分发和上下文隔离难以实现

2.3 DeepAgents 的解决方案



DeepAgents 通过引入"类人"的工作流解决了这些问题：

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              DeepAgents 类人工作流                              │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                                │
│    1. 计划 (Plan)          ──→   生成任务清单 (Todo)                           │
│          │                                                                     │
│          ▼                                                                     │
│    2. 执行 (Execute)       ──→   调用工具完成任务                              │
│          │                                                                     │
│          ▼                                                                     │
│    3. 记忆 (Memory)        ──→   存入文件系统                                  │
│          │                                                                     │
```

```
│              ▼                                                    │
│   4. 委托 (Delegate)      ──→  子代理处理复杂子任务                │
│                                                                   │
└──────────────
```


3.3 详细场景对比表

场景类型	能力说明	工作逻辑/技术特点	代表性案例
深度调研 与报告生成	支持长周期、多步骤、多来源信息整合的研究任务	• 自动生成研究计划（Todo） • 调用搜索工具获取资料 • 将关键信息写入文件系统 • 使用子代理深入研究子课题 • 主代理统一规划、整合结果	LangChain Deep Research OpenAI Deep Research
自动编程 与代码助理	理解代码、修改代码、生成新文件、执行工具链	• 代理可读写虚拟文件系统 • 自动分析源码并输出 diff • 人工审批保证安全写入 • 调用 Shell/测试工具执行 • 项目规范写入 /memories	LangChain DeepAgents CLI Anthropic Claude Code Manus
复杂流程 自动化	将多个步骤串联为可控流程，适合企业级自动化任务	• 任务分解 → 多步骤规划 • 搜索、筛选、处理、生成 • 文件系统存储中间数据 • 多工具、多子任务并行	DeepAgents 求职助手 自动生成分析报告 客服知识库构建

3.4 场景选择建议

```
┌──────────────
```

```
│  ✓ 任务时间跨度长（分钟级/小时级）                                  │
│  ✓ 需要多角色协作                                                  │
│                                                                    │
│  不需要使用 DeepAgents 的场景：                                     │
│  ✗ 简单问答                                                        │
│  ✗ 单轮对话                                                        │
│  ✗ 无需工具调用的任务                                              │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

四、本章小结

核心要点

1. DeepAgents 是什么？

- 基于 LangChain 和 LangGraph 的企业级高级智能体框架
- 是 LangGraph 的"最佳实践实现"
- 成品级框架，开箱即用

2. 解决的核心问题

- 规划能力缺失
- 遗忘与混乱
- 环境交互困难
- 上下文污染
- 协作编排复杂

3. 适用场景

- 深度调研
- 全栈代码生成
- 复杂数据分析
- 自动化运维
- 复杂工作流编排

课后思考

1. 你当前的业务场景是否适合使用 DeepAgents？
2. 传统 Agent 开发的五大问题中，你遇到过哪些？
3. 你期望 DeepAgents 帮助解决哪些具体问题？

本课件基于《DeepAgents_框架介绍与应用实践》第一阶段内容整理